

In this case, the browser will not share the cookie information for the requests that are not from the same origin. This means any request other

than the server from which the cookie is received will be treated as NOT a candidate for sending the domain cookie. All such requests are expected

to have 401 kinds of response. This way, strict cookie policies can be enforced by the application server depending on the need of making and accepting the cross-site requests.

- **Samesite=lax**

This is a more relaxing setting and is a default one if not explicitly set by the server. With this setting, any cross-site requests will not have cookies sent but same-site requests will have a cookie added by the browser. Classic cases can be requested by the sub-domain of the TLD (top-level domain).

- **Samesite=none**

This is the most relaxing configuration on the cookie. The cookie will be shared for any cross-site request by the browser. The only mandate is to have a cookie set with a 'secure' flag. The product owner can choose and select the combination of SameSite attributes and the CORS policy to attain freedom on cross-origin requests.

Note: The developer or the owner of the code should make effective use of CORS and SameSite for maximum security. For example, setting SameSite to strict while devising CORS to allow all arbitrary origins to make a request with the application server will be conflicting. So, the mitigation mechanisms used must be carefully devised.

CORS: Figure 5 describes the concepts related to CORS.

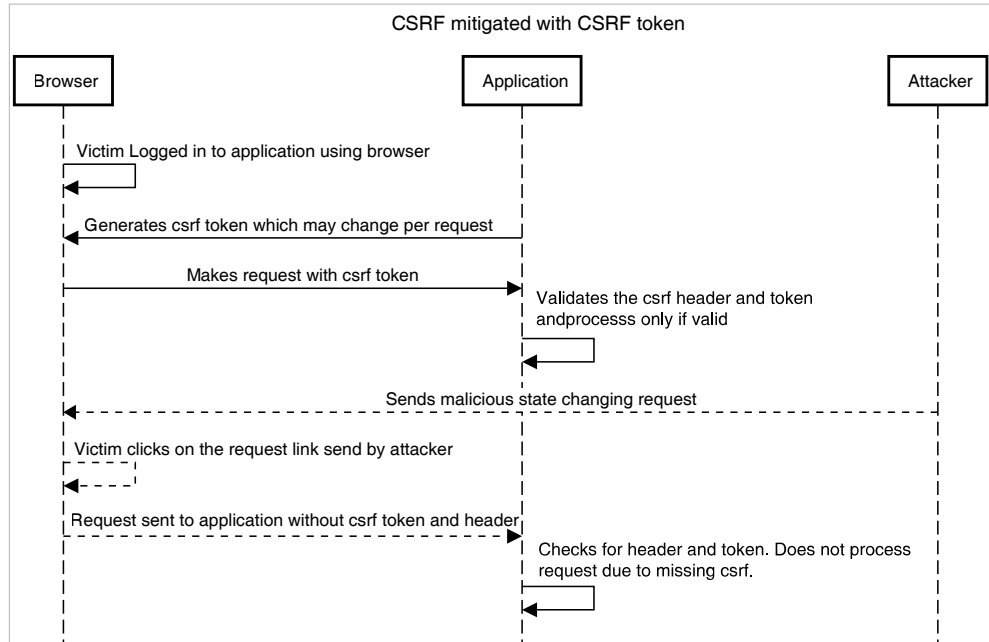


Figure 3: Mitigating CSRF with a CSRF token

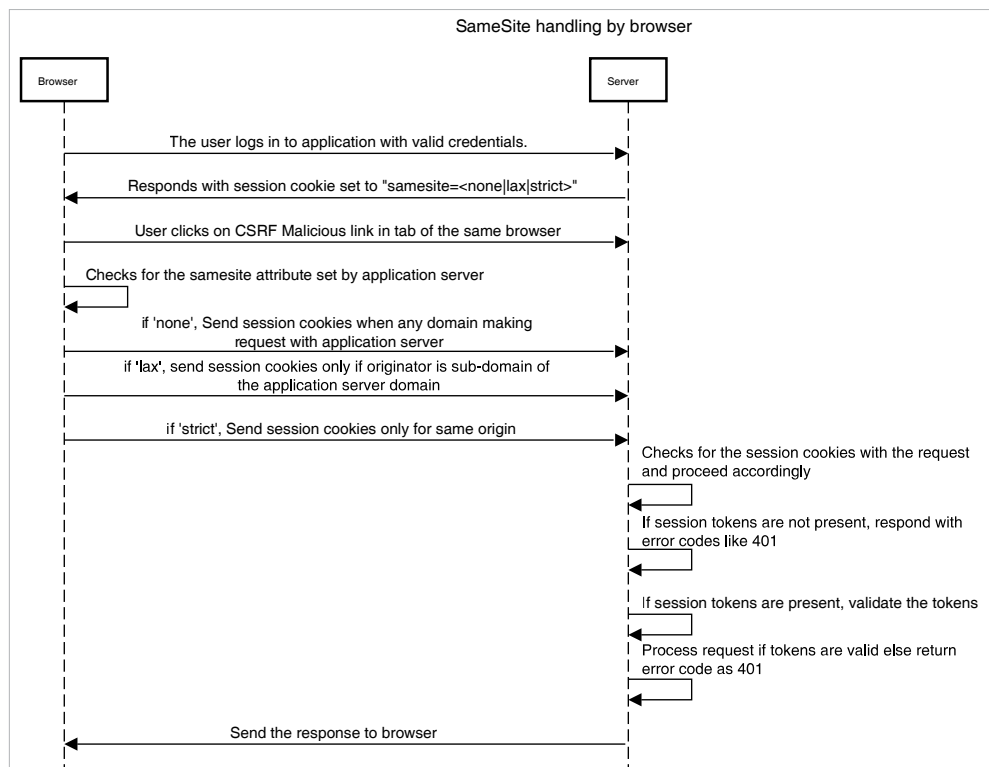


Figure 4: SameSite handling by the browser